

Kubernetes Networking

Kubernetes Production · Lesson 13

Built from the official Kubernetes documentation

kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Explain the **Kubernetes network model** and its fundamental requirements
- Describe the role of **CNI plugins** (Calico, Cilium, Flannel)
- Explain how **kube-proxy** implements **Services**
- Resolve names with **cluster DNS** (CoreDNS) and its naming scheme
- Write **NetworkPolicy** manifests to isolate and allow traffic

Reference: *Cluster Networking* — kubernetes.io/docs/concepts/cluster-administration/networking

13.1 The Kubernetes Network Model

Four Networking Concerns

Kubernetes networking addresses **four distinct concerns**:

- **Container-to-container** — solved by Pods sharing `localhost`
- **Pod-to-Pod** — the **primary focus** of the network model
- **Pod-to-Service** — stable virtual IPs, covered by Services
- **External-to-Service** — exposing workloads, also covered by Services

Instead of dynamic port allocation, **every Pod gets its own cluster-wide IP address** — you never link Pods or map container ports to host ports.

Fundamental Requirements

The model imposes requirements that hold **without NAT**:

- Every **Pod** gets its own unique, cluster-wide **IP address**
- Pods on a node can communicate with **all Pods on all nodes** without NAT
- **Agents on a node** (kubelet, system daemons) can reach all Pods on that node

This lets Pods be treated much like **VMs or physical hosts** for port allocation, naming, and service discovery.

CKA tip: no source NAT is applied to Pod-to-Pod traffic — a Pod's own IP is the same address other Pods use to reach it.

Non-Overlapping IP Ranges

Clusters must allocate **non-overlapping** IP addresses for three spaces:

Range	Configured by
Pod IPs	Network plugin (CNI)
Service IPs (cluster IPs)	kube-apiserver
Node IPs	kubelet / cloud-controller-manager

- Supports **IPv4-only**, **IPv6-only**, or **dual-stack** clusters
- All components must agree on the configured **primary IP family**
- Overlapping ranges break routing and Service resolution

13.2 CNI Plugins

The Role of CNI

The network model is **implemented by the container runtime** on each node, which uses **Container Network Interface (CNI)** plugins.

- Vanilla Kubernetes ships **no default CNI** — you choose and install one
- The plugin assigns Pod IPs and wires up Pod-to-Pod connectivity
- Capabilities range from **basic** add/remove of interfaces to **advanced IPAM**, multi-plugin chaining, policy, and encryption

Without a working CNI installed, Pods stay in **Pending / ContainerCreating** and CoreDNS Pods never become Ready.

Choosing a CNI → see Lesson 5

Which CNI to run and how to configure it (Calico / Cilium / Flannel / Kube-OVN..., CIDRs, encapsulation) is a **provisioning** decision — covered in **Lesson 5 — Container Runtimes & CNI**.

Here, the only plugin trait that matters is **NetworkPolicy** support:

- **NetworkPolicy is enforced by the CNI** — **Calico, Cilium, Antrea** enforce it
- **Flannel** alone does **not** — policy objects are created but **ignored**

CKA tip: exam clusters come with a CNI installed; you focus on the **model**, **DNS** and **NetworkPolicy** — choosing/installing the CNI is Lesson 5.

13.3 kube-proxy & Services

kube-proxy and Services

A **Service** gives a stable virtual IP (**cluster IP**) and name in front of a changing set of Pods.

- **kube-proxy** runs on each node and watches Services and EndpointSlices
- It programs **network rules** (iptables or IPVS) on the node
- Traffic to a Service's cluster IP is **load-balanced** to backend Pod IPs
- It is **optional** — some CNIs (e.g. Cilium) can replace kube-proxy

Pod IPs are ephemeral; **always reach Pods through a Service**, never by a hard-coded Pod IP.

Service Types (Quick Recap)

Type	Reachable from	Use
ClusterIP	Inside the cluster only	Default; internal apps
NodePort	<NodeIP>:<port>	Simple external access
LoadBalancer	External LB IP	Cloud-provider exposure
ExternalName	DNS CNAME	Alias to an external host

- Pod-to-Service traffic flows through kube-proxy's rules to a backend Pod
- External-to-Service traffic typically enters via NodePort or LoadBalancer

13.4 Cluster DNS (CoreDNS)

Cluster DNS with CoreDNS

CoreDNS runs as a cluster add-on (in `kube-system`) and gives Services and Pods resolvable names.

- The kubelet points every Pod's `/etc/resolv.conf` at the DNS Service
- Default **cluster domain**: `cluster.local`
- Services get **A/AAAA** records; named ports get **SRV** records
- Resolution is namespace-aware via a DNS **search path**

CKA tip: if name resolution fails, check the **CoreDNS Pods** and the `kube-dns` Service in `kube-system`.

The Naming Scheme

A **Service** is resolvable as:

```
my-svc.my-namespace.svc.cluster-domain.example
```

- Same namespace: just `my-svc`
- Cross-namespace: `my-svc.my-namespace`
- Fully qualified: `my-svc.my-namespace.svc.cluster.local`

A **Pod** by IP is resolvable as:

```
172-17-0-3.my-namespace.pod.cluster.local
```

SRV for a named port: `_port-name._port-protocol.my-svc.my-namespace.svc.cluster.local`

DNS Search Path & Lookup

The kubelet writes a search path so short names expand automatically:

```
search <namespace>.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

Verify resolution from a Pod:

```
# Resolve a Service by its FQDN
kubectl run dns-test --rm -it --image=busybox:1.28 --restart=Never -- \
  nslookup nginx.web.svc.cluster.local

# Short name (same namespace) – search path completes it
kubectl exec -it dns-test -- nslookup nginx
```


13.5 NetworkPolicy

NetworkPolicy Basics

NetworkPolicy controls traffic at **L3/L4** (IP / port) for TCP, UDP, SCTP between Pods and other network entities.

- It is a **namespaced** resource (`networking.k8s.io/v1`)
- Requires a **CNI that supports NetworkPolicy** (Calico, Cilium, Antrea...)
- By **default** Pods are non-isolated — **all** ingress and egress is allowed
- A Pod becomes **isolated** for a direction once a policy selects it
- Policies are **additive** — the result is the **union** of matching rules

With Flannel alone, NetworkPolicy objects are **created but not enforced**.

Anatomy of a Policy

Key fields in `spec`:

- **podSelector** — which Pods the policy applies to (empty `{}` = all in ns)
- **policyTypes** — `Ingress`, `Egress`, or both
- **ingress.from / egress.to** — allowed peers via:
 - **podSelector** — Pods in the same namespace
 - **namespaceSelector** — Pods in selected namespaces
 - **ipBlock** — `cidr` with optional `except`
- **ports** — `protocol` + `port` the rule allows

`podSelector` **and** `namespaceSelector` in **one** entry = logical **AND**
(specific Pods in specific namespaces); separate entries = **OR**.

Default-Deny All Ingress

An empty `podSelector` selects **every** Pod in the namespace; with no `ingress` rules, **all** inbound traffic is dropped:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: default
spec:
  podSelector: {}
  policyTypes:
    - Ingress
```

A common pattern: apply **default-deny**, then add explicit **allow** policies.

Allow From a Specific Pod

Permit ingress to `role: db` Pods only from `role: frontend` Pods on TCP `6379`:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: frontend
      ports:
        - protocol: TCP
          port: 6379
```

Allow From a Namespace

Permit ingress from **all** Pods in namespaces labeled `project: myproject`:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-namespace
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          project: myproject
  ports:
  - protocol: TCP
    port: 6379
```

Restrict Egress

Allow `role: db` Pods to send traffic only to a CIDR block on TCP `5978`:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-egress-cidr
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Egress
  egress:
  - to:
    - ipBlock:
        cidr: 10.0.0.0/24
    ports:
    - protocol: TCP
      port: 5978
```

For a connection to succeed, the **source's egress** *and* the **destination's ingress** must both allow it.

Key Takeaways

- The network model: **every Pod gets a unique cluster-wide IP**; Pod-to-Pod works **without NAT**
- **CNI plugins** (Calico, Cilium, Flannel) implement the network — vanilla K8s ships none by default
- **kube-proxy** programs node rules so **Services** load-balance to Pods
- **CoreDNS** resolves `my-svc.<ns>.svc.cluster.local`; a search path expands short names
- **NetworkPolicy** is namespaced, default-deny-friendly, and **requires a supporting CNI** to be enforced

End of Lesson 13

Next: Lesson 14 — RBAC (Security)

```
kubectl get networkpolicy · kubectl get svc -n kube-system · kubectl exec -it pod -- nslookup <svc>
```